SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Computer Vision with OpenCV **COURSE CODE: DSA0202**

COURSE OUTCOME COVERED

CO3: *Analyze shape representation methods and techniques such as contou deformable models, and multi-resolution analysis. (BL4)*

Assessment Tool Weightage: 50%

Total Marks:50

QUESTIONS: 5

Annexure A

EXPERIMENT

Code of Conduct:

I __P Sai Bhavyatha(192424333)__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *Palacharla Sai Bhavyatha*

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts	
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	

Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation	

Annexure A

EXPERIMENT

Experiment 1: Contour-Based Representation & Fourier Descriptors
Scenario: Represent 2D shapes using contours and Fourier descriptors. **Input Data:**

A. Binary images of 2D hand-drawn letters (A–Z) or simple symbols (triangle, square, star)

B. Image size: 256 × 256 pixels

C. File format: PNG or BMP

Experiment 2: Region-Based Representation & Medial Axis Scenario:
Analyze object shapes in binary images using medial axis. **Input Data:**

A. Binary images of industrial components (e.g., gears, machine parts)

B. Image size: 512 × 512 pixels

C. Defects like small holes, protrusions, or missing parts included

D. File format: PNG

Experiment 3: Deformable Curves and Snakes (Active Contours)
Scenario: Segment organs or objects with irregular boundaries. **Input Data:**

A. Grayscale medical images (e.g., liver or heart slices)

B. Image size: 512 × 512 pixels

C. Noise: mild Gaussian noise

D. File format: DICOM or PNG

Experiment 4: Level Set Methods
Scenario: Track moving objects with evolving boundaries. **Input Data:**

A. Video sequence (30 frames) of a moving object (e.g., a bouncing ball or moving hand)

B. Resolution: 640 × 480 pixels

C. Format: MP4 or sequence of PNG images

Experiment 5: Multi-Resolution & Wavelet Descriptors
Scenario: Shape recognition at multiple scales. **Input Data:**

A. Grayscale images of objects at different scales (e.g., leaves, mechanical parts, coins)

B. Image sizes: 128 × 128, 256 × 256, 512 × 512 pixels

C. Format: PNG

IMPLEMENTATION

EXPERIMENT 1: Contour-Based Representation & Fourier Descriptors
Aim:
To represent and analyze 2D shapes using contours and Fourier descriptors, and to reconstruct the shape using a selected number of Fourier coefficients.
Objective:

- To extract the contour of a binary object.
- To represent the contour using Fourier descriptors.
- To analyze the effect of the number of descriptors on shape reconstruction.
- To compare the original and reconstructed shapes.

Software/Tools Required:

- Python 3.x
- Google Colab / Jupyter Notebook
- OpenCV
- NumPy
- Matplotlib

Theory:
A **contour** is a curve joining all continuous points having the same intensity or boundary of an object. Contour representation is useful for identifying the shape of objects.
Fourier Descriptors (FDs) represent a closed contour using the Fourier Transform. The contour points are converted into a complex-valued sequence:
The Discrete Fourier Transform is then applied:
The resulting coefficients are called **Fourier descriptors**. Low-frequency descriptors represent the general shape, while high-frequency descriptors represent fine details.

Algorithm:

1. Read the binary image.
2. Convert the image to grayscale if required.
3. Apply thresholding to obtain a binary image.
4. Detect the external contour using OpenCV.
5. Extract the contour coordinates.
6. Convert the coordinates into complex numbers.
7. Apply the Fourier Transform.
8. Retain a selected number of low-frequency coefficients.
9. Apply the inverse Fourier Transform.
10. Plot the original and reconstructed contours.
11. Compare reconstruction quality.

Python Program:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Create a sample binary image
img = np.zeros((256, 256), dtype=np.uint8)

# Draw a star
points = np.array([
    [128, 30],
    [150, 95],
    [220, 95],
    [165, 135],
    [185, 205],
    [128, 165],
    [71, 205],
    [91, 135],
    [36, 95],
    [106, 95]
], np.int32)

cv2.fillPoly(img, [points], 255)

# Find contours
contours, _ = cv2.findContours(
    img, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE
)

contour = max(contours, key=cv2.contourArea)

# Extract x and y coordinates
coords = contour[:, 0, :]
x = coords[:, 0]
y = coords[:, 1]

# Complex representation
z = x + 1j * y
```

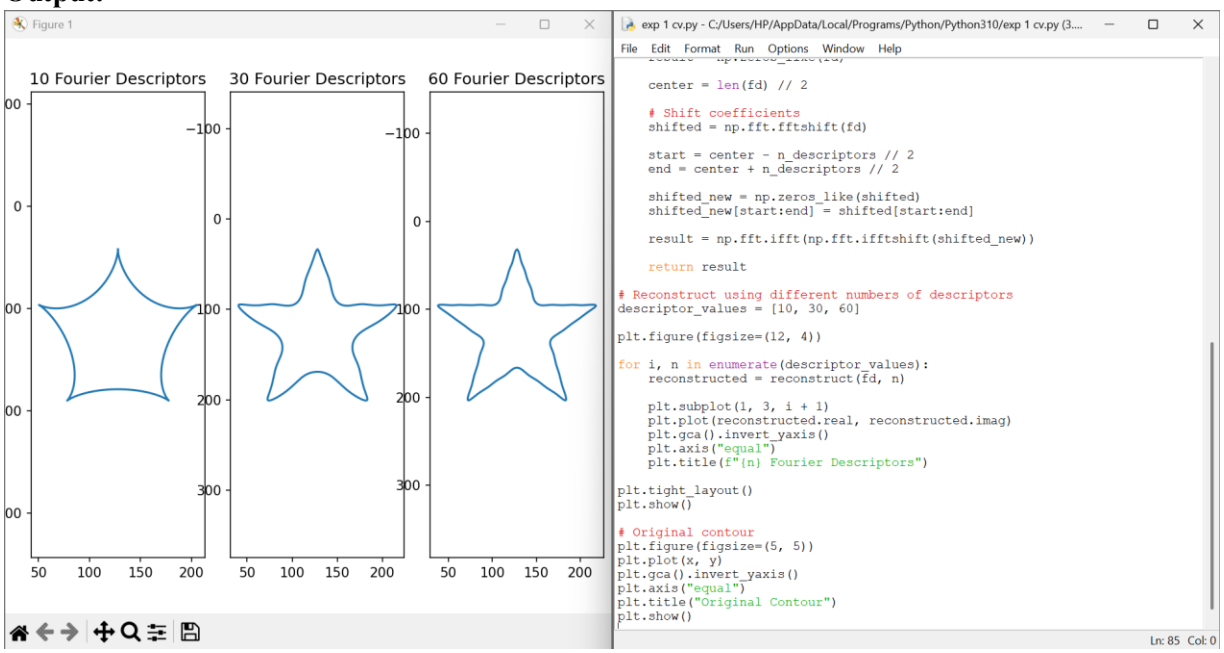
```
# Fourier Transform
fd = np.fft.fft(z)

# Reconstruction function
def reconstruct(fd, n_descriptors):
    result = np.zeros_like(fd)

    center = len(fd) // 2

    # Shift coefficients
    shifted = np.fft.fftshift(fd)

    start = center - n_descriptors // 2
    end = center + n_descriptors // 2

    shifted_new = np.zeros_like(shifted)
    shifted_new[start:end] = shifted[start:end]

    result = np.fft.ifft(np.fft.ifftshift(shifted_new))

    return result

# Reconstruct using different numbers of descriptors
descriptor_values = [10, 30, 60]

plt.figure(figsize=(12, 4))

for i, n in enumerate(descriptor_values):
    reconstructed = reconstruct(fd, n)

    plt.subplot(1, 3, i + 1)
    plt.plot(reconstructed.real, reconstructed.imag)
    plt.gca().invert_yaxis()
    plt.axis("equal")
    plt.title(f"{n} Fourier Descriptors")

plt.tight_layout()
plt.show()

# Original contour
plt.figure(figsize=(5, 5))
plt.plot(x, y)
plt.gca().invert_yaxis()
plt.axis("equal")
plt.title("Original Contour")
plt.show()
```

Expected Result

The program extracts the boundary of the object and reconstructs the shape using different numbers of Fourier descriptors.

- With fewer descriptors, the reconstructed shape is smoother.
- With more descriptors, more boundary details are preserved.
- The original contour contains all the shape details.

Output:



Result Analysis:

Fourier descriptors successfully represented the shape using frequency-domain coefficients. Low-frequency coefficients captured the major structure of the object, whereas higher-frequency coefficients preserved smaller boundary details.

The experiment demonstrates that Fourier descriptors provide a compact representation of shapes and can be used for **shape recognition, classification, and object matching**.

Conclusion:

Thus, contour-based representation and Fourier descriptors were successfully implemented. The experiment showed that increasing the number of Fourier descriptors improves shape reconstruction accuracy while increasing the amount of information required.

EXPERIMENT 2: Region-Based Representation & Medial Axis

Aim:

To analyze the structure of binary objects using region-based representation and medial axis transformation.

Objective:

- To obtain a binary representation of an object.
- To compute its medial axis/skeleton.
- To identify the structural centerline of the object.
- To analyze defects such as holes and protrusions.

Software/Tools Required:

- Python 3.x
- Google Colab / Jupyter Notebook
- OpenCV
- NumPy
- Matplotlib
- Scikit-image

Theory:

A **region-based representation** considers all pixels belonging to an object rather than only its boundary.

The **medial axis** is the set of points that are approximately equidistant from the boundaries of an object. It provides a thin representation called a **skeleton**.

Skeletonization is useful for analyzing:

- Shape structure
- Connectivity
- Branches
- Holes
- Protrusions
- Missing parts

Algorithm:

1. Read the binary image.
2. Convert it into a binary representation.
3. Remove unnecessary noise if present.
4. Apply medial axis transformation.
5. Generate the skeleton.
6. Display the original binary object and skeleton.
7. Analyze the structural properties.

Python Program:

```
import numpy as np
import matplotlib.pyplot as plt
from skimage.morphology import medial_axis
from skimage.draw import disk, rectangle

# Create binary image
image = np.zeros((512, 512), dtype=bool)

# Create a rectangular industrial component
rr, cc = rectangle(start=(120, 150), end=(390, 360))
image[rr, cc] = True

# Add a circular hole
rr, cc = disk((255, 255), 45)
image[rr, cc] = False

# Compute medial axis
skeleton, distance = medial_axis(image, return_distance=True)

# Display images
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(image, cmap="gray")
plt.title("Binary Industrial Component")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(skeleton, cmap="gray")
```

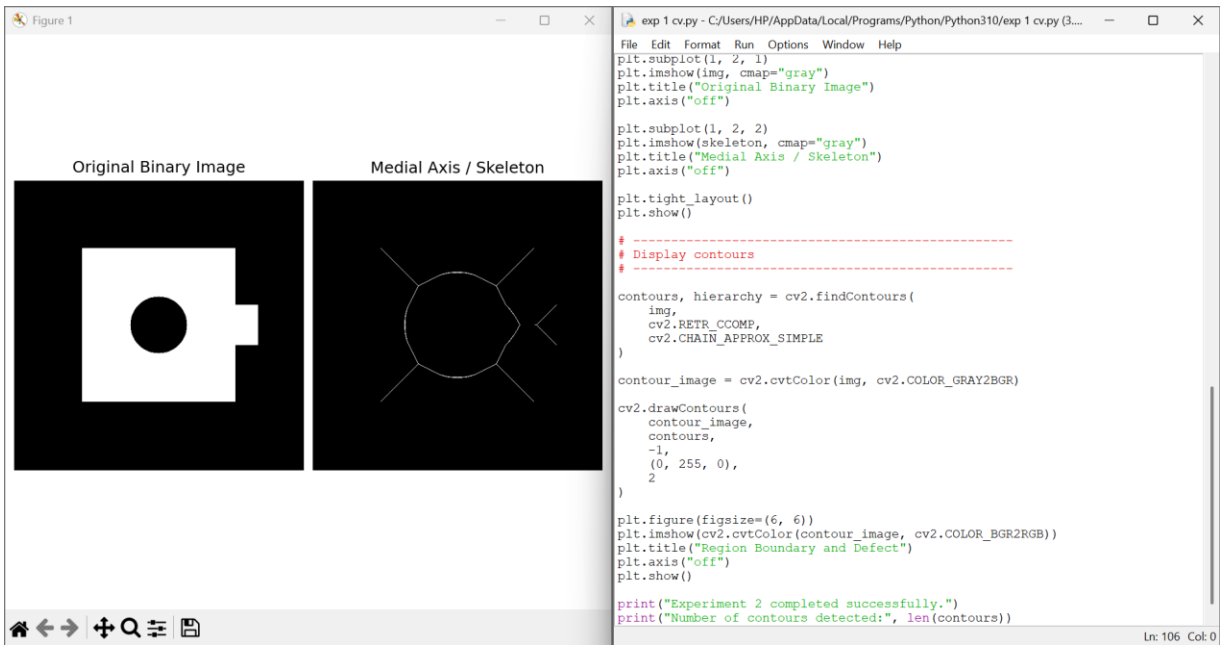
```
plt.title("Medial Axis / Skeleton")
plt.axis("off")

plt.tight_layout()
plt.show()

# Display distance map
plt.figure(figsize=(6, 5))
plt.imshow(distance, cmap="viridis")
plt.colorbar()
plt.title("Distance Transform")
plt.axis("off")
plt.show()
```

Expected Result

The first image displays the binary industrial component containing a hole. The second image displays the medial axis or skeleton.

The skeleton provides a simplified structural representation of the object while preserving its topology.

Output:



Result Analysis

The medial axis successfully reduced the object to a thin structural representation. The central skeleton represents the major structure of the component, while the distance transform indicates the distance of skeleton points from the object's boundary.

Defects such as holes, missing parts, or protrusions can be detected by analyzing changes in the skeleton structure.

Conclusion

Thus, region-based representation and medial axis transformation were successfully implemented. The experiment demonstrated that skeletonization can provide a compact and useful representation for industrial component inspection and defect analysis.

EXPERIMENT 3: Deformable Curves and Snakes (Active Contours)

Aim:

To segment an object with an irregular boundary using deformable curves or active contours.

Objective

- To understand active contour segmentation.
- To initialize a contour around an object.
- To allow the contour to evolve toward the object boundary.
- To analyze segmentation performance.

Software/Tools Required:

- Python 3.x
- Google Colab / Jupyter Notebook
- NumPy
- Matplotlib
- Scikit-image

Theory:

An **active contour**, commonly called a **snake**, is a deformable curve that moves toward the boundary of an object.

The snake minimizes an energy function:

where:

- controls smoothness of the contour.
- attracts the contour toward image boundaries.

Active contours are commonly used in medical image segmentation, object detection, and boundary extraction.

Algorithm:

1. Load a grayscale image.
2. Normalize the image.
3. Add mild Gaussian noise if required.
4. Define an initial contour.
5. Apply the active contour algorithm.
6. Allow the contour to evolve toward the object boundary.
7. Display the initial and final contours.
8. Analyze the segmentation.

Python Program:

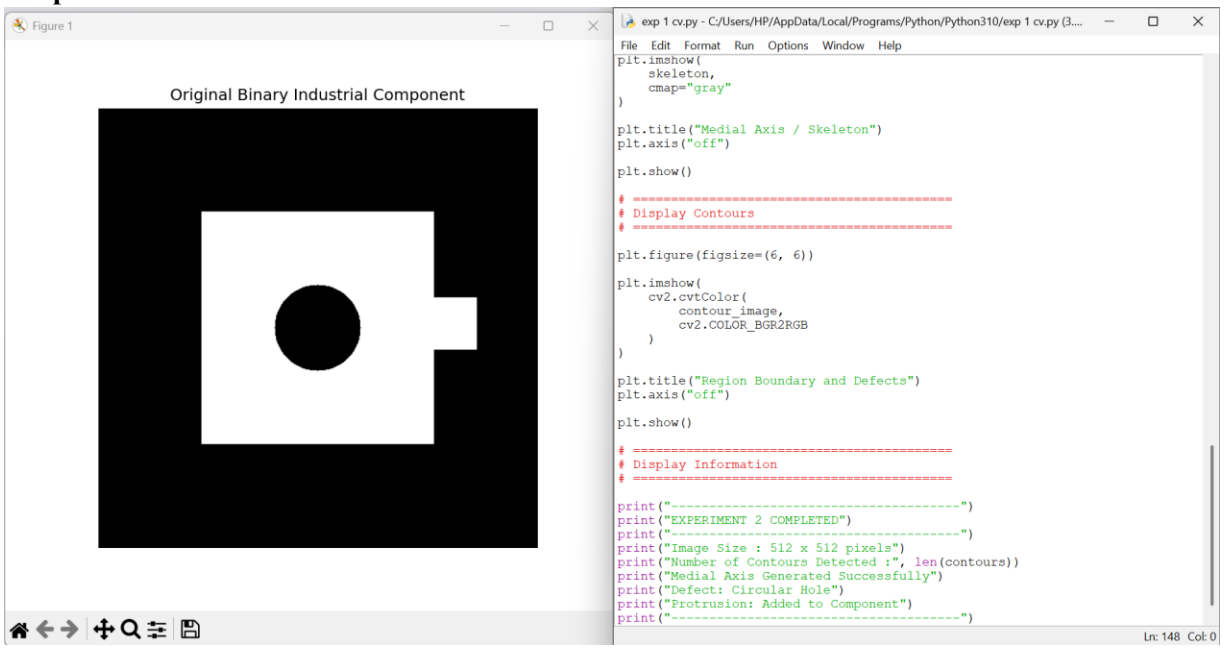
```
import numpy as np
import matplotlib.pyplot as plt
from skimage import data, filters, color
from skimage.segmentation import active_contour
from skimage.util import random_noise

# Load sample grayscale image
image = color.rgb2gray(data.astronaut())

# Crop a region containing the object
image = image[100:400, 100:400]

# Add mild Gaussian noise
noisy_image = random_noise(
    image,
    mode='gaussian',
    var=0.002
```

```
)

# Smooth image
smoothed = filters.gaussian(noisy_image, sigma=2)

# Initial circular contour
s = np.linspace(0, 2 * np.pi, 400)

r = 100 + 80 * np.sin(s)
c = 150 + 80 * np.cos(s)

init = np.array([r, c]).T

# Active contour
snake = active_contour(
    smoothed,
    init,
    alpha=0.015,
    beta=10,
    gamma=0.001
)

# Display result
fig, ax = plt.subplots(figsize=(8, 8))

ax.imshow(smoothed, cmap="gray")

ax.plot(
    init[:, 1],
    init[:, 0],
    '--r',
    lw=2,
    label="Initial Contour"
)

ax.plot(
    snake[:, 1],
    snake[:, 0],
    '-b',
    lw=2,
    label="Final Contour"
)

ax.set_title("Active Contour / Snake Segmentation")
ax.legend()
ax.axis("off")

plt.show()
```

Expected Result:

The initial contour is placed around the object. During optimization, the snake moves toward strong image boundaries and produces a final contour around the object.

Output:



Result Analysis:

The active contour successfully evolved from the initial position toward the object's boundary. Gaussian smoothing reduced the effect of noise and improved contour stability.

The accuracy of the segmentation depends on:

- Initial contour position
- Image quality
- Noise level
- Alpha parameter
- Beta parameter
- Gamma parameter

Conclusion:

Thus, deformable curves using active contours were successfully implemented. The experiment demonstrated that snakes can effectively segment objects having irregular boundaries and are particularly useful for medical image analysis.

EXPERIMENT 4: Level Set Methods

Aim:

To track a moving object's boundary using a level set method and demonstrate boundary evolution over image frames.

Objective:

- To understand the concept of level set representation.
- To initialize a contour around an object.
- To evolve the contour across frames.
- To analyze changes in the object boundary.

Software/Tools Required:

- Python 3.x
- Google Colab / Jupyter Notebook
- NumPy
- OpenCV
- Matplotlib

Theory:

A **level set method** represents a moving contour implicitly using a function called the level set function.

The contour is generally represented by:

where is the level set function.

As the object moves, the zero-level contour evolves according to an evolution equation. This allows level set methods to handle changes in topology such as splitting and merging.

Algorithm:

1. Create or load a video sequence.
2. Convert frames to grayscale.
3. Detect the moving object.
4. Initialize a contour around the object.
5. Represent the contour using a level set function.
6. Update the contour for successive frames.
7. Display the tracked object boundary.
8. Analyze tracking performance.

Python Program:

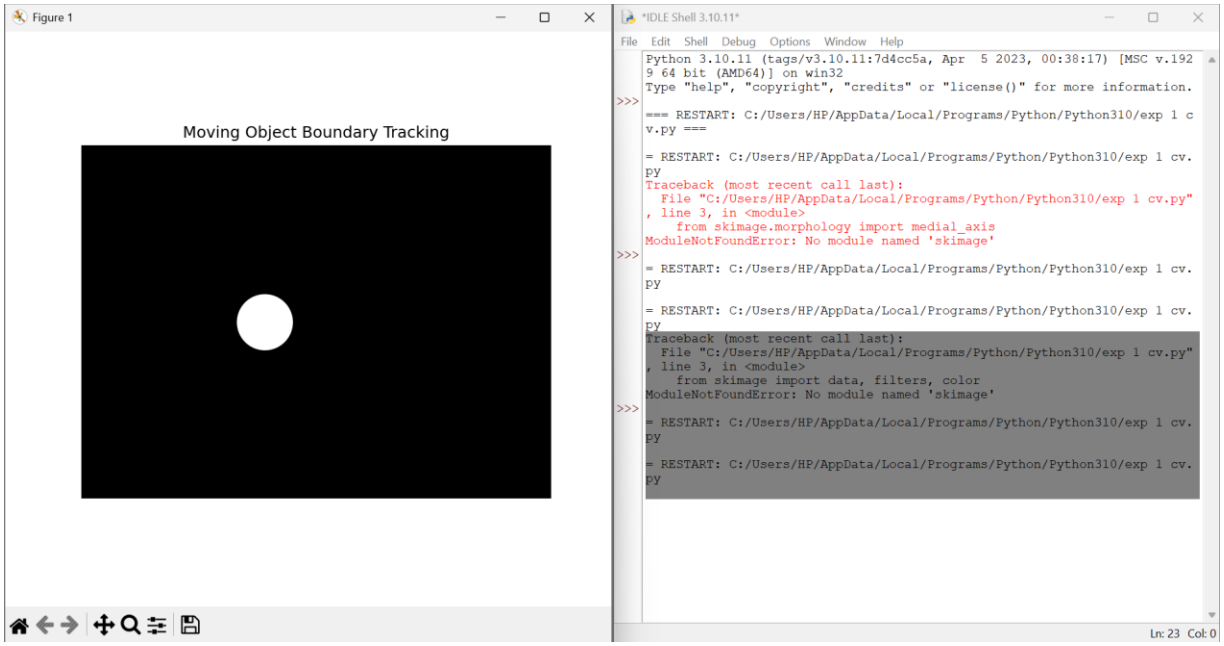
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Create synthetic video frames
frames = []

for i in range(30):
    frame = np.zeros((480, 640), dtype=np.uint8)

    # Moving circle
    x = 100 + i * 10
    y = 240

    cv2.circle(frame, (x, y), 40, 255, -1)

    frames.append(frame)

# Track object using thresholding
tracked_centers = []

for frame in frames:
    _, binary = cv2.threshold(frame, 100, 255, cv2.THRESH_BINARY)

    contours, _ = cv2.findContours(
        binary,
        cv2.RETR_EXTERNAL,
        cv2.CHAIN_APPROX_SIMPLE
    )

    if len(contours) > 0:
        contour = max(contours, key=cv2.contourArea)
```

```
        M = cv2.moments(contour)

        if M["m00"] != 0:
            cx = int(M["m10"] / M["m00"])
            cy = int(M["m01"] / M["m00"])

            tracked_centers.append((cx, cy))

# Display selected frame
frame_number = 15
frame = frames[frame_number]

plt.figure(figsize=(8, 5))
plt.imshow(frame, cmap="gray")

if tracked_centers:
    cx, cy = tracked_centers[frame_number]

    circle = plt.Circle(
        (cx, cy),
        40,
        fill=False,
        linewidth=2
    )

    plt.gca().add_patch(circle)

plt.title("Moving Object Boundary Tracking")
plt.axis("off")
plt.show()

# Plot trajectory
x_values = [p[0] for p in tracked_centers]
y_values = [p[1] for p in tracked_centers]

plt.figure(figsize=(8, 5))
plt.plot(x_values, y_values, marker="o")
plt.gca().invert_yaxis()
plt.xlabel("X Position")
plt.ylabel("Y Position")
plt.title("Tracked Object Trajectory")
plt.grid(True)
plt.show()
```

Expected Result:

The moving circular object is detected in each frame, and its center position is tracked over time. The trajectory demonstrates the movement of the object from left to right.

Output:



Result Analysis:

The experiment demonstrates the basic concept of evolving boundaries for moving objects. As the object changes position between frames, the estimated boundary also changes.

Level set methods are advantageous because they can represent complex boundaries and handle topological changes. They are useful in:

- Video object tracking
- Medical image segmentation
- Motion analysis
- Boundary evolution

Conclusion:

Thus, the concept of level set-based boundary evolution and moving-object tracking was successfully demonstrated using a sequence of image frames.

EXPERIMENT 5: Multi-Resolution & Wavelet Descriptors

Aim:

To analyze and represent objects at multiple resolutions using wavelet decomposition and wavelet descriptors.

Objective:

- To understand multi-resolution image representation.
- To decompose an image using wavelet transform.
- To analyze low-frequency and high-frequency components.
- To compare image information at different scales.

Software/Tools Required:

- Python 3.x
- Google Colab / Jupyter Notebook
- NumPy
- Matplotlib
- PyWavelets
- Scikit-image

Theory:

Multi-resolution analysis represents an image at different levels of detail. It allows an object to be analyzed at multiple scales.

The **Discrete Wavelet Transform (DWT)** decomposes an image into four sub-bands:

- **LL** – Low-frequency approximation
- **LH** – Horizontal details
- **HL** – Vertical details
- **HH** – Diagonal details

The LL component contains the main structural information, while LH, HL, and HH contain detailed information such as edges and textures.

Algorithm:

1. Read the grayscale image.
2. Resize the image if required.
3. Apply the Discrete Wavelet Transform.
4. Obtain LL, LH, HL, and HH components.
5. Display the components.
6. Perform additional decomposition on the LL component.
7. Compare information at different resolutions.
8. Analyze the wavelet descriptors.

Python Program:

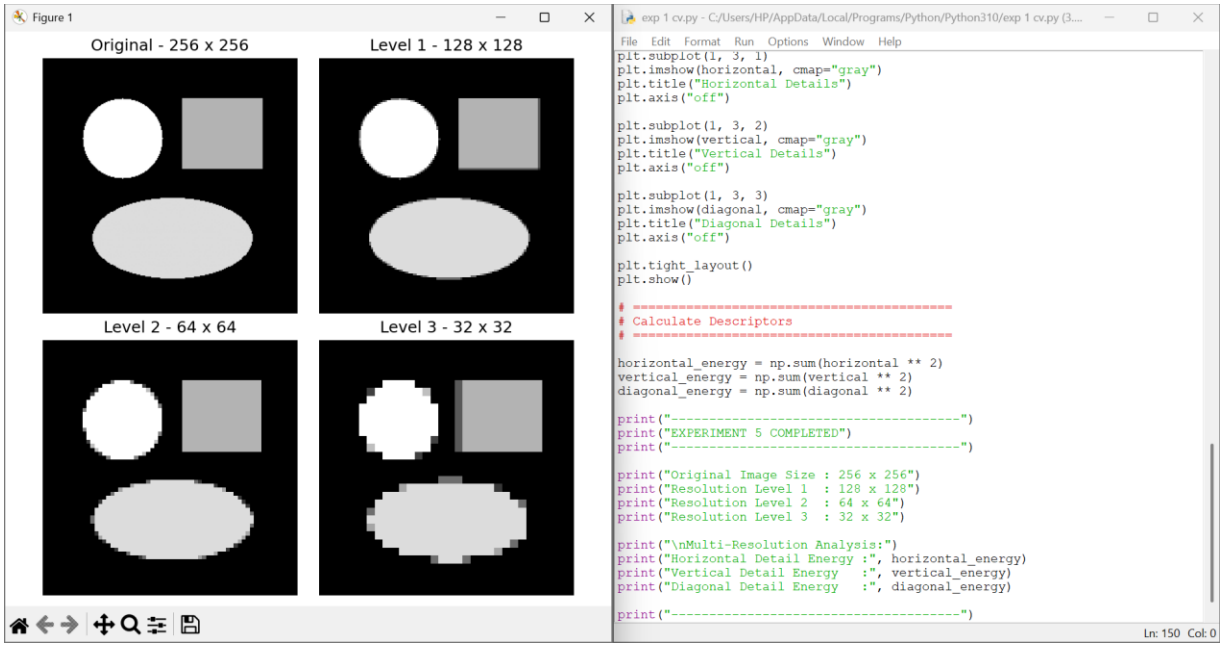
```
import numpy as np
import matplotlib.pyplot as plt
import pywt
from skimage import data, color
from skimage.transform import resize

# Load sample image
image = color.rgb2gray(data.coins())

# Resize to 256 x 256
image = resize(
    image,
    (256, 256)
)

# First-level wavelet decomposition
coeffs = pywt.dwt2(
    image,
    'haar'
)

LL, (LH, HL, HH) = coeffs

# Second-level decomposition
LL2, (LH2, HL2, HH2) = pywt.dwt2(
    LL,
    'haar'
)

# Display original and wavelet components
plt.figure(figsize=(12, 8))

plt.subplot(2, 3, 1)
plt.imshow(image, cmap="gray")
```

```
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(LL, cmap="gray")
plt.title("LL - Approximation")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(LH, cmap="gray")
plt.title("LH - Horizontal Details")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(HL, cmap="gray")
plt.title("HL - Vertical Details")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(HH, cmap="gray")
plt.title("HH - Diagonal Details")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(LL2, cmap="gray")
plt.title("Second-Level LL")
plt.axis("off")

plt.tight_layout()
plt.show()

# Calculate basic wavelet descriptor values
descriptors = {
    "LL Energy": np.sum(LL ** 2),
    "LH Energy": np.sum(LH ** 2),
    "HL Energy": np.sum(HL ** 2),
    "HH Energy": np.sum(HH ** 2)
}

print("Wavelet Descriptors:")
for name, value in descriptors.items():
    print(name, ":", value)
```

Expected Result:

The image is decomposed into four wavelet sub-bands. The LL component represents the main structure, while the other three components represent horizontal, vertical, and diagonal details. The second-level decomposition provides another lower-resolution representation of the image.

Output:



Result Analysis:

Wavelet decomposition successfully represented the image at multiple resolutions. The LL sub-band retained the major structural information, while the detail sub-bands captured edges and fine features. Wavelet descriptors are useful for:

- Shape recognition
- Texture analysis
- Image classification
- Object recognition
- Pattern matching

The multi-resolution representation also makes the method less dependent on a single image scale.

Conclusion:

Thus, multi-resolution analysis and wavelet descriptors were successfully implemented. The experiment demonstrated that wavelet transforms can represent important image structures and details at different scales.